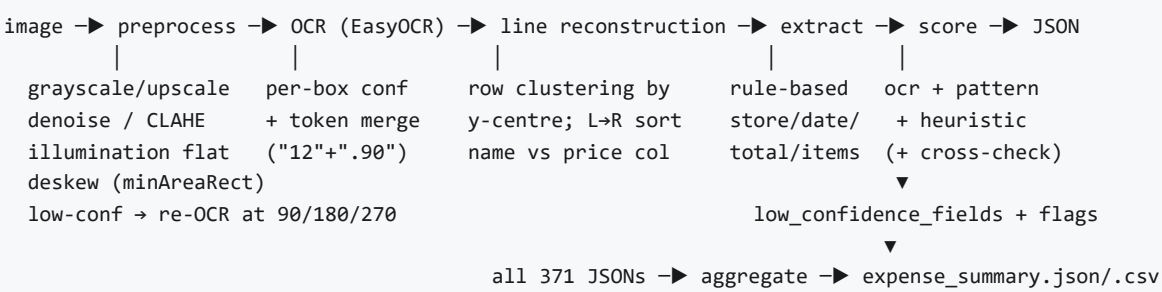
Receipt OCR & Confidence-Aware Extraction (Writeup)

Carbon Crunch ML Ops assignment. A deterministic batch pipeline that turns receipt images into per-field, **confidence-scored** JSON plus an aggregate expense summary. Python 3.13, CPU-only, no runtime network calls (EasyOCR downloads its models once).

1. Approach

Pipeline



Why rule-based key-information extraction

The dataset ships **no ground-truth labels**, so a supervised KIE model (LayoutLMv3, Donut) cannot be trained or validated within the time budget. Instead every field is selected by transparent, config-driven heuristics (`config.yaml` holds all keyword lists and thresholds, no magic numbers in code), tuned against a 30-receipt hand-labelled eval set. This keeps the system debuggable and lets the confidence layer, rather than a black box, carry the reliability signal.

Confidence model

Per field, `confidence` in `[0,1]` is a weighted mean of up to four signals; when a signal does not apply (e.g. no items, so no cross-check) its weight is dropped and the rest are renormalised.

Signal	Meaning
`ocr`	mean OCR confidence of the tokens that form the value
`pattern`	format validity: date parses & plausible (1.0 / 0.5 if day-order ambiguous / 0.0); money matches `^\d+\.\d{2}$` in range; store = alphabetic-char ratio
`heuristic`	strength of the rule that fired: tier-A total keyword 1.0, tier-B 0.7, fallback 0.3; store suffix/vendor-match 1.0, top-scored 0.6
`cross_check`	**total only**: `1.0` when `abs(item-price-sum - total)` is within `max(5%*total, RM 0.50)` , else `(min/max)` squared

Field	ocr	pattern	heuristic	cross_check
store_name	0.35	0.20	0.45	n/a
date	0.35	0.40	0.25	n/a
total_amount	0.35	0.20	0.20	0.25
item_name	0.70	0.30	n/a	n/a
item_price	0.55	0.45	n/a	n/a

Reliability handling. Fields below 0.7 are listed in `low_confidence_fields` (with granular `items[i].price` entries). Flags: `missing_{store,date,total}`, `no_items`, `total_from_fallback`, `conflicting_total` (2+ disagreeing tier-A candidates, chosen value kept with a 0.15 penalty, rejects recorded in `meta.alternatives`), `date_order_ambiguous`, `items_price_sum_mismatch` (cross-check < 0.6), `items_truncated`, `low_mean_ocr_conf`, `unreadable_image`, `ocr_empty`, `pipeline_error:<type>`.

2. Tools used

Tool	Why
OpenCV	grayscale/upscale, fastNIMeans denoise, illumination flattening (divide by large-Gaussian background), CLAHE, `minAreaRect` deskew
EasyOCR	primary OCR: gives a per-box probability (the `ocr` signal) and is robust on faded thermal print without hand-tuned thresholds
Tesseract (pytesseract)	optional comparison baseline + OSD orientation; swappable behind the same `OcrResult` interface
rapidfuzz	`token_set_ratio` for run-wide vendor-name canonicalisation and fuzzy eval scoring
python-dateutil	tolerant date parsing (`dayfirst=True`) to ISO `YYYY-MM-DD`
pydantic v2	frozen config tree (rejects unknown keys) and the `{value, confidence}` output schema
pandas / matplotlib	expense-summary CSV and the confidence-calibration bar chart

3. Results

Coverage/confidence are from the full 367-image run (`make batch`); the accuracy and calibration rows are `make eval` against the 30 hand-labelled receipts in `eval/labels.csv`.

- **Coverage** (non-null over 371 receipts): store **100.0 %**, date **84.5 %**,

total **95.6 %**, items **63.5 %**. Mean field confidence: store 0.84, date 0.77, total 0.77, item_name 0.69, item_price 0.88. Full 367-image run, 0 pipeline errors, 0.10 img/s on CPU

(`outputs/run_report.md`).

- **Eval accuracy** (n = 30 labelled): store exact 47 % / fuzzy >= 90 **70 %**;

date exact **80 %**; total +/- 0.05 **37 %**; n_items exact 37 % / +/- 1 **77 %**. The low total figure is the honest weak spot: on GST receipts the ranker still sometimes takes *Total (Excluding GST)*, the GST-summary line, or (on fast-food bundles) the first item price instead of *Total Inclusive of GST*; almost every such miss carries `total_from_fallback` plus a sub-0.7 confidence.

- **Calibration** (`eval/calibration.png` , pooled store/date/total, n = 90):

accuracy climbs monotonically with the confidence bucket, **0 %** (0 to 0.5), **44 %** (0.5 to 0.7), **66 %** (0.7 to 0.9), **89 %** (0.9 to 1.0), so the score is a usable reliability signal even where extraction is wrong.

- **Engine comparison** (`eval/engine_comparison.md` , 30-image eval set):

EasyOCR is the default (the Tesseract binary was not installed on the build machine). Enabling the preprocessing pipeline raised mean OCR confidence **0.73 to 0.76** and mean field confidence **0.77 to 0.80**, at a cost of **+4.8 s/image** (13.8 to 18.6 s/image on CPU). Preprocessing is kept on.

4. Challenges faced

- **Layout diversity**: the 371 receipts span Malaysian SROIE vendors and

US-style chains (Walmart, Dollar Tree, and others) with different totals wording and column layouts; the tier-A/tier-B keyword lists are config-tunable to absorb this.

- **Subtotal vs grand total**: `SUBTOTAL` contains the substring `TOTAL` , so it

is hard-excluded first; tier-A phrasings like "*Total Inclusive of GST*" legitimately contain `GST` , which would otherwise be excluded, so tier-A overrides the GST/TAX exclusion. Among equal tiers the **last** occurrence wins.

- **Uneven lighting / thermal fade**: illumination flattening + CLAHE recover

contrast; measured +0.03 OCR confidence in the ablation.

- **Ambiguous `dd/mm` dates**: parsed `dayfirst=True` ; when both components are

<= 12 the order is unprovable, so `pattern` confidence drops to 0.5 and the `date_order_ambiguous` flag fires.

- **No ground truth**: built a 30-row eval set by hand-labelling evenly-sampled

receipts; every accuracy/calibration number in §3 comes from it.

5. Improvements

- **LayoutLMv3 / Donut** for KIE once labels exist, with an active-learning loop

seeded by the current `low_confidence_fields`.

- **Per-vendor templates** keyed off the canonicalised store name (fixed field

positions once a vendor is recognised).

- **Learned confidence calibrator** (isotonic / Platt) fitted on the eval set to

replace the hand-set weights.

- **Fine-tune the recognizer** on receipt fonts; benchmark PaddleOCR; enable GPU.
- **Currency-aware summary**: detection is wired (`meta.currency_detected`) but

EasyOCR often drops the faint `$` / `RM` glyph; a dedicated symbol detector would make multi-currency totals first-class.